

Homework 5

1. Objective

The midterm's Step 3 (Hybrid Analysis) trained a centralized RandomForest over TF-IDF-encoded APIcall sequences and standardized network-traffic features to classify a malware sample's family. This assignment repeats that task but replaces the machine-learning method with Federated Learning (FL): three simulated clients train a shared model collaboratively without ever pooling their raw samples, using the Flower framework's FedAvg strategy. The centralized baseline is retained and re-measured so the federated numbers have something concrete to compare against.

2. Dataset & Feature Pipeline

The dataset is the midterm's CAPEv2 sandbox output: 5 usable labelled samples (after dropping runs with fewer than 50 API calls) spanning 3 families plus one unknown family sample held aside. Each sample carries a full APIcall sequence from the CAPE monitor and 8 numeric network-traffic features (UDP/TCP/ICMP/HTTP counts, DNS query count, unique destination ports/IPs, malicious-domain hits).

Because gradient-based training needs more than 5 training instances, each sample's API sequence is cut into overlapping fixed-length windows (500 calls, 250-call stride), producing roughly 174 window instances from the 5 samples. This raises the instance count but not the independent sample count. Every train/test split is therefore made at the sample level, before windowing, and independently verified at runtime by `data_utils.assert_no_leakage()`.

3. Why the Midterm's Pipeline Had to Change for Federated Learning

Two components of the midterm pipeline are fundamentally incompatible with federated learning, which constrained the redesign:

- RandomForest cannot be federated

FedAvg aggregates a global model by averaging client parameter vectors, $w_{\text{global}} = \sum_k (n_k/n) \cdot w_k$. A RandomForest has no such vector, it is a set of discrete tree structures (split thresholds, feature indices, leaf values) with no meaningful arithmetic mean across two forests grown on different data. Federated learning therefore constrains the model class to differentiable, parametric models. The forest is replaced with HybridCNN, a small convolutional network whose weights can be legally averaged.

- TF-IDF cannot be fitted federatedly either

Fitting TF-IDF requires corpus-wide document frequencies (i.e. seeing every client's data on one machine, exactly what FL forbids). It is replaced with a fixed, data-independent API-name vocabulary (`build_vocab.py`) which is the set of API names that can ever appear is fixed by the CAPEv2 monitor's hook configuration, a property of the sandbox rather than of the training data, so every client can derive the identical vocabulary locally with zero cross-client communication. The same objection applies to a fitted StandardScaler for the network features; it is replaced with a data-independent log1p compression (a fixed divisor, not a fitted statistic).

4. Model Architecture

HybridCNN preserves the midterm's early-fusion design, replacing the fitted feature extractors with learned, federatable equivalents:

Stage	Operation	Role
API view	Embedding(vocab→32) → Conv1d(32→64,k=5) → Conv1d(64→64,k=5) → AdaptiveMaxPool1d	Learned analogue of the midterm's TF-IDF unigram/bigram features
Network view	Linear(8→16) + ReLU	The 8 log1p-compressed traffic features

Fusion	Concatenate (64+16=80-d) → Dropout → Linear(80→64) → Linear(64→3)	Same early-fusion strategy as the midterm's hybrid RandomForest
--------	---	---

Trainable parameters: 41,011 (≈ 160 KB in fp32) — the quantity that must be uploaded and downloaded by every client on every FedAvg round, i.e. the model's communication cost.

5. Federated Learning Setup

Three clients are simulated in-process with Flower's simulation engine (`run_simulation.py`, `client.py`, `server.py`). Per training fold:

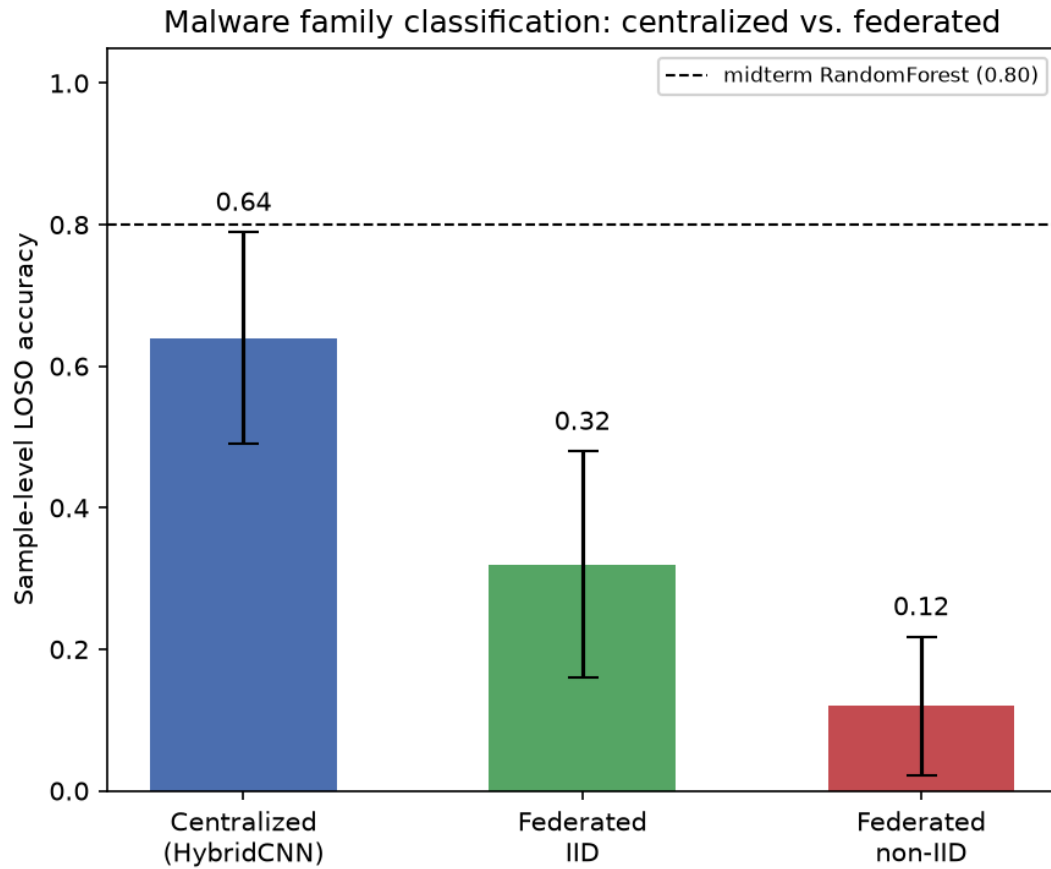
- Partitioning: the training samples are split across the 3 clients at the sample level (never at the window level, for the same leakage reason as Section 2), in two regimes:
 - IID, samples are shuffled and dealt round-robin.
 - Non-IID, each client is specialized toward one malware family, the realistic scenario of three independent security vendors each seeing a different slice of the threat landscape.
- Local training: each client trains HybridCNN on its own shard for 2 local epochs per round, using the exact same `train_epochs()` routine the centralized baseline uses, so any accuracy gap is attributable to federation itself and not to a different optimizer.
- Class weighting: computed from each client's own local windows only — a client cannot see the global class distribution without leaking counts to the server.
- Aggregation: FedAvg with full participation (all 3 clients selected every round) for 40 rounds. A custom `SavingFedAvg` strategy retains the final aggregated global weights, since Flower's simulation API does not otherwise expose the trained model.

6. Evaluation Protocol

Identical to the midterm's, so the CNN and the RandomForest are directly comparable: leave-onesample- out (LOSO) cross-validation. With 5 usable samples this gives 5 folds; the model is re-initialized and (for the federated runs) the training set re-partitioned fresh each fold. Per-window predictions on the held-out sample are collapsed to one verdict by majority vote. Because one fold is worth 0.20 accuracy and weight initialization alone can move the result across that range, every configuration is run under 5 different seeds and reported as mean $\pm$ std, never as a single run.

7. Results

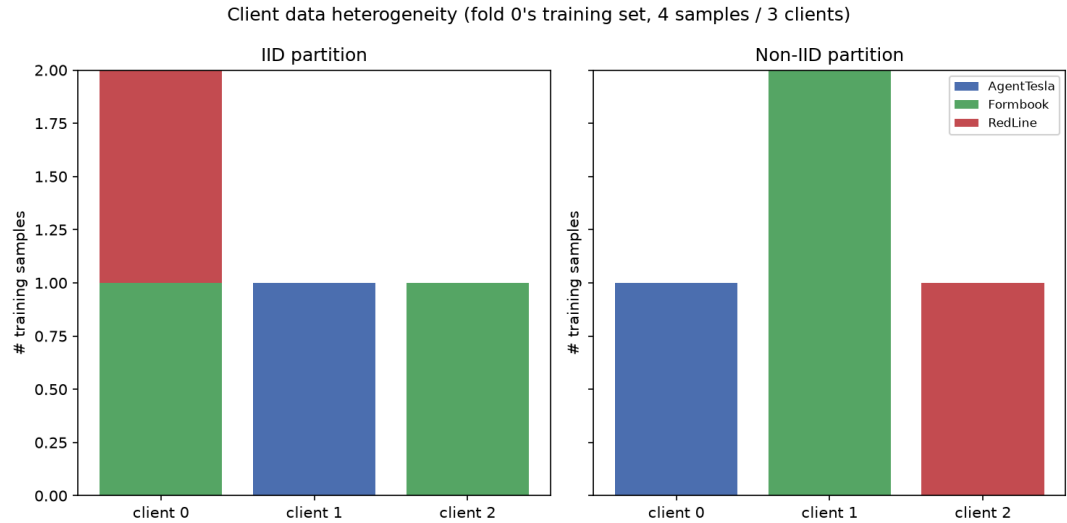
Configuration	Sample-level accuracy	Window-level accuracy
Centralized HybridCNN	0.64 ± 0.15	0.57 ± 0.04
Federated, IID partition	0.32 ± 0.16	0.28 ± 0.16
Federated, non-IID partition	0.12 ± 0.10	0.11 ± 0.07
Midterm RandomForest	0.80	-



Federation costs real accuracy here, and the non-IID regime costs substantially more than the IID one. The centralized model reaches 80% of the RandomForest's accuracy, federated IID reaches 40% of it, federated non-IID reaches only 15%.

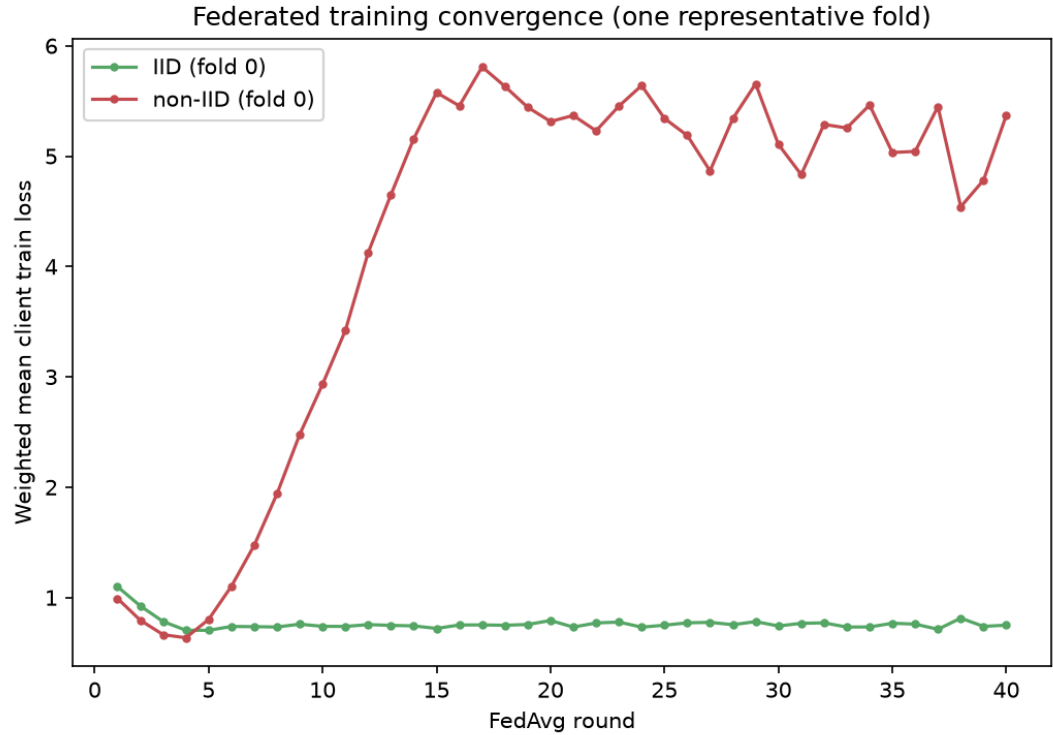
7.1. Client data heterogeneity

With only 4 training samples split across 3 clients per fold, every partition is forced into near-single-family shards. The IID partition gets there by chance of the shuffle (e.g. shard sizes [2,1,1]), the non-IID partition by design (each client owns one family).



7.2. Client drift and convergence

Under IID partitioning, loss settles quickly near 0.7 and stays flat. Under non-IID partitioning, loss rises through round ≈ 20 and then oscillates around 5 for the remainder of training — it never converges. This is the classic FedAvg client-drift failure mode: each client's local update pulls the shared model toward its own single family, and because the three families are disjoint, three rounds of local training pull the global model in three different, largely uncorrelated directions. Averaging those updates partially cancels the very learning each client just did, and the next round starts the fight over again from a worse position.



8. Communication Cost

HybridCNN's 41,011 parameters (≈ 160 KB in fp32) must be uploaded and downloaded by every client on every round. At $3 \text{ clients} \times 40 \text{ rounds}$, one fold's federated run transmits $\approx 3 \times 40 \times 160 \text{ KB} \approx 19 \text{ MB}$ of parameter traffic total (up+down) — compared to zero raw-sample bytes leaving any client, versus the centralized baseline's requirement that every client's raw API traces and network features be pooled onto one machine before training can begin at all. This is the actual trade being made: substantial roundtrip parameter traffic and, on this dataset, a real accuracy cost, in exchange for the raw malware traces never leaving the client that captured them.

9. Limitations

- $N=5$ samples is not enough to draw general conclusions about FL vs. centralized malware classification

It is enough to demonstrate the mechanism (client drift under extreme data scarcity) but not to estimate how a production-scale federation with thousands of samples per client would perform. The 0.20-accuracy granularity of a single

LOSO fold, and the fact that one held-out family (RedLine) is unwinnable regardless of method, both stem directly from dataset size.

- Partitioning 4 samples across 3 clients cannot realize a "true" IID regime

The IID partition in Section 7.1 is IID in procedure (random shuffle) but not in outcome (near-single-family shards are almost unavoidable at this scale). The IID vs. non-IID gap measured here is therefore a lower bound on what client heterogeneity would cost at production scale, not an upper bound.

- 40 rounds may still be short of full convergence for the non-IID regime specifically; Section 7.2's loss curve had not stabilized by round 40. More rounds were not run because the IID/IID-vs-non-IID comparison, not absolute non-IID performance, is the object of this assignment.

10. Conclusion

Federated learning is achievable on this task once the midterm's non-federatable components (RandomForest, TF-IDF, StandardScaler) are replaced with differentiable, data-independent equivalents (HybridCNN, a fixed API vocabulary, log1p scaling). On this 5-sample dataset, federation has a measurable and explainable accuracy cost relative to the centralized baseline ($0.64 \rightarrow 0.32 \rightarrow 0.12$ for centralized $\rightarrow$ federated-IID $\rightarrow$ federated-non-IID), driven by client drift under FedAvg when clients hold disjoint, single-family data — a mechanism directly visible in the training-loss trajectories (Figure 3) and traceable to the dataset's small size (Figure 2) rather than to any defect in the federated protocol itself.